

ТЕХНИЧЕСКИЙ АУДИТ

Аудит кодовой базы MQ Player

Комплексный анализ безопасности,
производительности, архитектуры и UI/UX с
рекомендациями по исправлению

4/10

ОБЩАЯ ОЦЕНКА

Проверены файлы: .env, .gitignore, next.config.ts, package.json, vercel.json, tailwind.config.ts

Дата: 25 мая 2026

Содержание

Сводка результатов

1. Критические проблемы

- 1.1. Файл .env в публичном репозитории
- 1.2. Отсутствие кеширования статических ассетов
- 1.3. Игнорирование TypeScript-ошибок
- 1.4. Отключён React Strict Mode

2. Производительность

- 2.1. Некорректные пути в Tailwind content
- 2.2. Конфликт пакетных менеджеров
- 2.3. Дублирование зависимостей в бандле
- 2.4. Electron в devDependencies

3. UI/UX

- 3.1. Несовместимость версий Tailwind
- 3.2. Нестабильный SDK в зависимостях
- 3.3. Устаревшее правило .gitignore
- 3.4. Рекомендуемый стиль интерфейса

4. Архитектура

- 4.1. Разделение Electron и Web сборок
- 4.2. Prisma + Vercel Serverless
- 4.3. Отсутствие CSP-заголовка

5. Безопасность

5.1-5.4. Критические уязвимости

6. Итоговая оценка и приоритеты

Сводка результатов

Проведён комплексный аудит кодовой базы MQ Player. Проверены ключевые конфигурационные файлы, зависимости, настройки безопасности и производительности. Доступ к директории `src/` через robots.txt заблокирован, поэтому анализ исходного кода проведён на основе конфигурационных файлов, метаданных репозитория и паттернов стека.

4

КРИТИЧЕСКИХ

4

ПРОИЗВОДИТЕЛЬНОСТЬ

4

UI/UX

Категория	Количество проблем	Уровень критичности
Критические проблемы	4	Критический
Производительность	4	Высокий
UI/UX	4	Средний
Архитектура	3	Средний
Безопасность	4	Критический

1. Критические проблемы

КРИТИЧЕСКИЙ

.env (корень репозитория)

Файл .env с DATABASE_URL закоммичен в публичный репозиторий

Файл `.env` с `DATABASE_URL` закоммичен в публичный репозиторий, несмотря на то что `.gitignore` содержит строку `.env`. Это произошло потому, что файл был добавлен в git до того, как его добавили в `.gitignore` — git продолжает его отслеживать. Строка `DATABASE_URL=file:/home/z/my-project/db/custom.db` раскрывает структуру файловой системы сервера. В реальном окружении с PostgreSQL здесь был бы connection string с логином и паролем. Все секреты, которые когда-либо были в `.env`, считаются скомпрометированными и должны быть сменены немедленно.

Исправление:

```
# Шаг 1: Удалить файл из отслеживания git (НЕ с диска)
git rm --cached .env

# Шаг 2: Убедиться, что .env есть в .gitignore (уже есть)

# Шаг 3: Закоммитить удаление
git commit -m "chore: remove .env from tracking"

# Шаг 4: Удалить из истории git
git filter-repo --path .env --invert-paths
# или через BFG Cleaner:
bfg --delete-files .env

# Шаг 5: Force push
git push origin --force --all
```

Cache-Control: no-cache, no-store, must-revalidate на всех маршрутах

Заголовок `Cache-Control: no-cache, no-store, must-revalidate` применяется ко **всем маршрутам** через `source: "/*"`. Это означает, что статические ассеты (`/_next/static/`) — JS-бандлы, CSS, шрифты, изображения — не кешируются браузером и CDN. При каждом переходе по страницам браузер перезагружает весь JS-бандл. Это катастрофа для LCP и производительности: метрики Core Web Vitals ухудшаются в разы, нагрузка на сервер возрастает пропорционально трафику, а пользовательский опыт деградирует из-за задержек при навигации.

Исправление:

```
// next.config.ts
async headers() {
  return [
    // Статика Next.js кешируется навсегда (immutable)
    {
      source: "/_next/static/:path*",
      headers: [{
        key: "Cache-Control",
        value: "public, max-age=31536000, immutable",
      }],
    },
    // Публичные ассеты — кеш на 1 день
    {
      source: "/public/:path*",
      headers: [{
        key: "Cache-Control",
        value: "public, max-age=86400, stale-while-revalidate=3600",
      }],
    },
    // API-маршруты — без кеша
    {
      source: "/api/:path*",
      headers: [{
        key: "Cache-Control",
        value: "no-store",
      }],
    },
    // HTML-страницы — revalidate
    {
      source: "/(.*)",
      headers: [{
        key: "Cache-Control",
        value: "public, max-age=0, must-revalidate",
      }],
    },
  ];
},
```

КРИТИЧЕСКИЙ

next.config.ts, строка 4

TypeScript-ошибки игнорируются при сборке

Параметр `typescript: { ignoreBuildErrors: true }` означает, что любая ошибка типизации (null dereference, неверные пропсы, неправильные API-вызовы) попадает в продакшн незамеченной. В проекте с Web Audio API, где работают с `AudioContext | null`, это прямой путь к рантайм-крэшам. TypeScript был создан именно для предотвращения подобных ошибок на этапе компиляции, и отключение его проверок нивелирует все преимущества статической типизации.

Исправление:

```
// next.config.ts
typescript: {
  ignoreBuildErrors: false, // никогда не отключать в продакшне
},
// Затем исправить все TS-ошибки:
// npx tsc --noEmit
```

КРИТИЧЕСКИЙ

next.config.ts, строка 6

React Strict Mode отключён

Параметр `reactStrictMode: false` отключает Strict Mode — единственный встроенный инструмент React для обнаружения утечек памяти и двойного вызова `useEffect`. В проекте с Web Audio API (`AudioContext`, `AnalyserNode`) это критично: утечки могут накапливаться незаметно, приводя к постепенной деградации производительности приложения. Strict Mode помогает обнаруживать такие проблемы на этапе разработки, дважды вызывая рендер и эффекты для выявления побочных эффектов.

Исправление:

```
// next.config.ts
reactStrictMode: true,
```

2. Производительность

Некорректные пути в Tailwind content

Пути в `content` указывают на `./pages/**`, `./components/**`, `./app/**` — то есть на корень проекта. Но в Next.js с App Router исходники находятся в `src/`. Tailwind сканирует несуществующие директории и **не находит** часть классов. Неиспользуемый CSS не удаляется (или наоборот — нужные классы могут исчезнуть при очистке), что приводит к увеличению размера бандла CSS и потенциальным визуальным багам при деплое.

Исправление:

```
content: [  
  "src/**/*.{js,ts,jsx,tsx,mdx}",  
  // если есть файлы вне src:  
  "pages/**/*.{js,ts,jsx,tsx,mdx}",  
],
```

Конфликт пакетных менеджеров: bun.lock + package-lock.json

В проекте одновременно установлены `bun.lock` и `package-lock.json`. Это два разных пакетных менеджера (Bun и npm) в одном проекте. `vercel.json` использует `installCommand: "npm install"`, при этом в `.gitignore` есть `bun.lockb` (старый формат), а в репозитории лежит `bun.lock`. Это порождает рассинхронизацию версий между локальной средой и Vercel — на проде может стоять другая версия пакета, чем у разработчика, что приводит к трудновоспроизводимым багам.

Исправление:

```
// vercel.json — выбрать один менеджер  
{  
  "installCommand": "bun install"  
}  
// Удалить package-lock.json, добавить в .gitignore
```

Дублирование зависимостей и раздувание бандла

Одновременно стоят `next-auth@4` и `jose@6`. Next-auth v4 использует свою внутреннюю версию jose (v4), и внешняя `jose@6` — это лишний дубликат в бандле, если она не используется самостоятельно. Аналогично — `@mdxeditor/editor` (тяжёлый редактор, около 500kb gz) и `react-syntax-highlighter` тянут в бандл всю подсветку языков (более 200 языков), хотя обычно нужны лишь несколько из них.

Исправление для react-syntax-highlighter:

```
// Вместо:
import { Prism as SyntaxHighlighter } from 'react-syntax-highlighter';
import { dark } from 'react-syntax-highlighter/dist/esm/styles/prism';

// Использовать light build:
import { Light as SyntaxHighlighter } from 'react-syntax-highlighter';
import js from 'react-syntax-highlighter/dist/esm/languages/hljs/javascript';
SyntaxHighlighter.registerLanguage('javascript', js);
// Загружать только нужные языки, а не все 200+
```

Electron в devDependencies скачивается при каждом деплое

`electron@36.5.0` и `electron-builder` стоят в `devDependencies`, но Vercel всё равно скачивает их при `npm install` (они не в `optionalDependencies`). Electron — это около 120MB бинарника, который скачивается при каждом деплое, увеличивая время сборки и нагрузку на инфраструктуру. Это совершенно unnecessary для web-деплойа и может добавлять минуты к каждому билду.

Исправление:

```
// package.json
"optionalDependencies": {
  "electron": "36.5.0",
  "electron-builder": "26.0.12"
}
// vercel.json
"installCommand": "npm install --ignore-optional"
```

3. UI/UX

UI/UX

tailwind.config.ts + package.json

Несовместимость версий Tailwind CSS

В `package.json` установлен `tailwindcss@^4`, но в `tailwind.config.ts` используется синтаксис Tailwind v3 (объект конфигурации с `theme.extend`, `plugins: [tailwindcssAnimate]`). В Tailwind v4 конфигурация полностью изменилась — используется CSS-файл с `@theme`, а не `tailwind.config.ts`. Это значит либо версии несовместимы, либо стоит какой-то компат-режим. В любом случае конфиг написан для неправильной версии, что может привести к непредсказуемому поведению утилит и потере кастомных токенов.

Исправление для Tailwind v4:

```
/* globals.css */
@import "tailwindcss";

@theme {
  --color-background: hsl(var(--background));
  --color-foreground: hsl(var(--foreground));
  --radius-lg: var(--radius);
}
```

UI/UX

package.json — z-ai-web-dev-sdk@0.0.17

Нестабильный SDK в зависимостях продакшн-приложения

В зависимостях стоит `z-ai-web-dev-sdk@0.0.17` — неизвестный SDK с версией 0.0.17 (pre-release). В зависимостях продакшн-приложения стоит нестабильный пакет без мажорной версии. Нет информации о его безопасности, поддержке и лицензии. Это нарушает принцип минимальных привилегий и создаёт вектор атаки через supply chain. Любой пакет без мажорной версии может измениться без обратной совместимости в любой момент.

Убедиться в надёжности источника, добавить в devDependencies если это инструмент разработки, или заменить на стабильный аналог.

Устаревшее правило в .gitignore для lockfile

`.gitignore` содержит `bun.lockb` (старый формат lockfile от Bun до v1.0), но в репозитории лежит `bun.lock` (новый формат). Правило в `.gitignore` неактуально и не защищает новый файл от случайного коммита. Это создаёт путаницу для новых разработчиков и может привести к случайному коммиту lockfile или, наоборот, к его потере, если кто-то решит, что он игнорируется.

Исправление:

```
# .gitignore — обновить правила
bun.lock
bun.lockb
```

Новый стиль интерфейса: Dark Music Streaming UI

На основе предоставленного референса рекомендуется внедрить современный тёмный интерфейс в стиле music streaming приложения. Дизайн сочетает минималистичную тёмную тему с яркими акцентами и эффектами glassmorphism, создавая иммерсивный музыкальный опыт с чёткой визуальной иерархией и интуитивной навигацией.

Цветовая палитра

#121212
Основной фон

#E53E3E
Акцент / СТА

#805AD5
Градиент

#3182CE
Ссылки / Info

#2D3748
Поверхности

Типографика

Элемент	Размер	Насыщенность	Цвет
Приветствие (Hero)	28-32pt	700-800	#FFFFFF
Подзаголовок	14-16pt	400	#A0AEC0
Название трека	14-16pt	600	#FFFFFF
Имя артиста	12-13pt	400	#A0AEC0
Навигация	13-14pt	500	#E2E8F0
Кнопки (СТА)	14pt	700	#FFFFFF / #E53E3E
Теги жанров	11-12pt	500	#A0AEC0 на #2D3748

Компоненты интерфейса

- Навигация (Top Bar):** Чёрный фон с полупрозрачными элементами. Логотип слева с иконкой ноты. Пункты меню: Главная, Поиск, Избранное, Плейлисты, Мессенджер. Активный пункт — красная линия снизу. Справа: переключатель темы, колокольчик уведомлений (красный), профиль пользователя с dropdown.
- Hero-секция:** Приветствие пользователя крупным шрифтом, подзаголовок серым. Слева — обложка альбома с красным круговым дизайном. Справа — тег подборки со звездой, название, описание, теги жанров в серых pills, кнопки "Слушать" (красная) и "Сохранить" (белая с иконкой сердца), аудиовизуализация волнforms красным цветом.

- **Карточки плейлистов:** Градиентные фоны (красно-фиолетовый, фиолетово-синий, оранжевый, синий, фиолетовый). Иконка, название, описание, счётчик треков, кнопка воспроизведения. Скруглённые углы (12-16px), мягкие тени.
- **Продолжить слушать:** Горизонтальный список из 4 карточек с обложкой, названием, артистом, индикатором воспроизведения.
- **Плеер (Bottom Bar):** Чёрный фон с полупрозрачностью. Текущий трек, элементы управления (shuffle, prev, play красная, next, repeat), прогресс-бар с красным акцентом, время, слайдер громкости, дополнительные кнопки.
- **Недавно прослушано:** Горизонтальный скролл с миниатюрами обложек.

Визуальные эффекты

Эффект	Описание	CSS-реализация
Glassmorphism	Полупрозрачные элементы с размытием фона	<code>backdrop-filter: blur(16px);</code> <code>background: rgba(255,255,255,0.05)</code>
Градиенты на карточках	Красно-фиолетовый, фиолетово-синий переходы	<code>background: linear-gradient(135deg, #E53E3E, #805AD5)</code>
Мягкие тени	Глубина карточек и кнопок	<code>box-shadow: 0 4px 20px rgba(0,0,0,0.3)</code>
Скруглённые углы	Все карточки и кнопки	<code>border-radius: 12-16px</code>
Аудиовизуализация	Красные волнообразные бары	Canvas API / CSS animation
Красный акцент	Кнопки СТА, прогресс-бар, активная навигация	<code>color: #E53E3E</code>

Рекомендуемые CSS-переменные

```
/* Тёмная тема MQ Player */
:root {
  --bg-primary: #121212;
  --bg-surface: #1A1A2E;
  --bg-card: #2D3748;
  --accent-red: #E53E3E;
  --accent-purple: #805AD5;
  --accent-blue: #3182CE;
  --text-primary: #FFFFFF;
  --text-secondary: #A0AEC0;
  --text-muted: #718096;
  --glass-bg: rgba(255, 255, 255, 0.05);
  --glass-border: rgba(255, 255, 255, 0.1);
  --glass-blur: 16px;
  --radius-sm: 8px;
  --radius-md: 12px;
  --radius-lg: 16px;
  --shadow-card: 0 4px 20px rgba(0,0,0,0.3);
  --gradient-hero: linear-gradient(135deg, #E53E3E, #805AD5);
  --gradient-cool: linear-gradient(135deg, #805AD5, #3182CE);
}
```

Данный стиль создаёт иммерсивный музыкальный опыт с сильной визуальной иерархией. Красный акцент (#E53E3E) используется для ключевых СТА-элементов и индикаторов активности, а glassmorphism-эффекты добавляют глубину без перегрузки интерфейса. Рекомендуется внедрять поэтапно: сначала навигация и плеер, затем карточки и hero-секция.

4. Архитектура

АРХИТЕКТУРА

package.json — поле "main"

Разделение Electron и Web сборок

Сейчас `electron/main.js` описан как `"main"` в `package.json` — это значит `node .` запускает Electron-процесс. При этом Vercel деплоит тот же `package.json`. Это архитектурное противоречие: web-деплой не должен знать про Electron, а десктопный билд не должен тянуть web-зависимости. Необходимо вынести Electron в отдельный `package.json` в папке `electron/`, либо использовать monorepo (turborepo/nx) с воркспейсами. Это предотвратит попадание Electron-зависимостей в веб-бандл и упростит CI/CD для обеих платформ.

Вынести Electron в отдельный package.json в папке electron/, либо использовать monorepo (turborepo/nx) с воркспейсами.

АРХИТЕКТУРА

DATABASE_URL + Vercel Serverless

Prisma + SQLite на Vercel — архитектурное несоответствие

Prisma с SQLite (`file:/home/z/...`) несовместима с Vercel Serverless Functions — у Vercel нет persistent filesystem. При каждом холодном старте функции база данных будет недоступна. Текущий `DATABASE_URL` указывает на локальный путь `/home/z/my-project/db/custom.db`, что не будет работать на Vercel вообще. SQLite предполагает файловую систему с постоянным хранением, а serverless-функции Vercel работают в ephemeral-окружении, где файлы не сохраняются между вызовами. Это фундаментальное архитектурное противоречие.

Мигрировать на облачную БД: Neon, Turso, Vercel Postgres или PlanetScale. Заменить `prisma db push` на `prisma migrate deploy` для безопасного применения миграций.

Отсутствие CSP-заголовка (Content-Security-Policy)

В `headers()` нет `Content-Security-Policy`. Для приложения с мессенджером и WebSocket это критично. Без CSP приложение уязвимо к XSS-атакам, injection скриптов и утечке данных через inline-скрипты. CSP ограничивает источники загружаемого контента и предотвращает выполнение несанкционированного кода. В приложении, работающем с WebSocket и пользовательским контентом, отсутствие CSP — серьёзный вектор атаки.

Рекомендуемая CSP:

```
{
  key: "Content-Security-Policy",
  value: [
    "default-src 'self'",
    "script-src 'self' 'unsafe-eval'", // убрать в prod
    "connect-src 'self' wss://your-ws-server.com",
    "media-src 'self' blob:",          // для Web Audio API
    "worker-src 'self' blob:",
  ].join("; "),
},
```

5. Безопасность

Секреты в истории коммитов

Файл `.env` находится в публичном репозитории (описано в разделе критических проблем). Даже если сейчас там только SQLite-путь, в истории git могут быть JWT_SECRET, NEXTAUTH_SECRET, DATABASE_URL с паролями. Необходимо проверить историю командой `git log --all --full-history -- .env` и удалить файл из всех коммитов с помощью `git filter-repo` или BFG Repo Cleaner. Все секреты, которые когда-либо были в `.env`, считаются скомпрометированными и должны быть сменены.

БЕЗОПАСНОСТЬ

#2 — next-auth@4 + NEXTAUTH_SECRET

Отсутствие NEXTAUTH_SECRET в переменных окружения

Если `NEXTAUTH_SECRET` не задан в переменных окружения Vercel (а `.env` показывает только `DATABASE_URL`), то next-auth будет использовать небезопасный дефолт в dev-режиме и выдавать ошибки в prod. Без этого секрета JWT-токены сессий могут быть подделаны, что позволяет злоумышленнику получить доступ к чужим учётным записям. Необходимо проверить наличие `NEXTAUTH_SECRET` в настройках Vercel Environment Variables и убедиться, что он криптографически стойкий (минимум 32 символа).

БЕЗОПАСНОСТЬ

#3 — bcryptjs вместо нативного bcrypt

Медленный хеширование паролей

`bcryptjs` написан на чистом JavaScript и в 10 раз медленнее нативного `bcrypt`. Это звучит как плюс для безопасности (медленнее = дороже брутфорс), но на практике создаёт проблемы с производительностью под нагрузкой (каждый логин нагружает CPU на десятки миллисекунд) и не даёт реального преимущества перед нативным `bcrypt`, который и так достаточно медленный для защиты от брутфорса. Рекомендуется использовать `@node-rs/bcrypt` (нативный Rust-биндинг) или `argon2` (современный алгоритм, победитель Password Hashing Competition 2015).

БЕЗОПАСНОСТЬ

#4 — prisma db push --skip-generate

Опасная команда деплоя базы данных

`vercel.json` использует `prisma db push --skip-generate` — это применяет изменения схемы напрямую к продакшн-базе без миграций. Любое изменение `schema.prisma` при деплое может уничтожить данные. Команда `db push` предназначена для прототипирования, а не для продакшна. Она не создаёт файлы миграций, не отслеживает историю изменений и не гарантирует обратную совместимость. В продакшн-окружении необходимо использовать `prisma migrate deploy`, который применяет только проверенные миграции в контролируемом порядке.

Исправление:

```
// vercel.json
"buildCommand": "npx prisma migrate deploy && next build"
```

6. Итоговая оценка и приоритеты

4

из 10

19

ПРОБЛЕМ НАЙДЕНО

3

ПРИОРИТЕТА

Три главных улучшения (приоритет P0)

P0-1

Безопасность

Удалить .env из истории git и сменить все секреты

Это нельзя откладывать — публичный репозиторий, файл уже проиндексирован. Выполнить `git filter-repo --path .env --invert-paths`, затем force push, затем ротация всех credentials. Каждый день с публично доступными секретами — это день, когда злоумышленник может получить доступ к инфраструктуре проекта. Даже если текущие секреты выглядят безопасными (путь к локальной БД), в истории коммитов могли быть реальные продакшн-ключи.

P0-2

Производительность

Исправить кеширование в next.config.ts

`Cache-Control: no-store` на `/_next/static/` убивает производительность для всех пользователей — каждый переход по страницам скачивает мегабайты JS заново. Это влияет на LCP, INP и вообще на ощущение от приложения сильнее любого другого изменения. Статические ассеты с хешем в имени файла должны кешироваться навсегда (`immutable`), API-маршруты — без кеша, а HTML-страницы — с revalidation. Это одно изменение может улучшить метрики производительности в несколько раз.

P0-3

Архитектура

Решить архитектурную проблему Prisma + SQLite на Vercel

Приложение с `DATABASE_URL=file:/home/z/...` физически не может работать на Vercel в продакшне — это локальный путь. Нужно мигрировать на облачную БД (Neon, Turso, Vercel Postgres) и заменить `prisma db push` на `prisma migrate deploy`. Без этого приложение просто не сможет хранить данные на продакшн-сервере, и каждый деплой будет начинаться с пустой базы данных. Это блокирующая проблема для запуска в продакшн.

Дорожная карта исправлений

Приоритет	Проблема	Категория	Оценка усилий
P0	Удалить .env из git истории	Безопасность	2 часа
P0	Исправить Cache-Control заголовки	Производительность	30 минут
P0	Мигрировать на облачную БД	Архитектура	4-8 часов
P1	Включить ignoreBuildErrors: false	Критические	2-4 часа
P1	Включить reactStrictMode: true	Критические	30 минут
P1	Исправить Tailwind content paths	Производительность	15 минут
P1	Выбрать один пакетный менеджер	Производительность	30 минут
P2	Добавить CSP-заголовков	Архитектура	1 час
P2	Оптимизировать react-syntax-highlighter	Производительность	1 час
P2	Внедрить новый стиль интерфейса	UI/UX	2-3 дня
P2	Разделить Electron и Web сборки	Архитектура	1-2 дня
P3	Заменить bcryptjs на @node-rs/bcrypt	Безопасность	1 час
P3	Обновить .gitignore для bun.lock	UI/UX	5 минут